

FDU 数据库 3. 数据库设计

本文参考以下教材:

- Database System Concepts (A. Silberschatz, H. Korth, S. Sudarshan) Chapter 6, 7
- 数据库系统概念 (A. Silberschatz, H. Korth, S. Sudarshan, 杨冬青等译) 第 6, 7 章

欢迎批评指正!

3.1 实体-联系模型

数据库设计的最初阶段是完整地描述用户的数据需求, 即**概念设计** (conceptual design) 阶段. 我们通常使用**实体-关系模型**来表示概念设计. 在**逻辑设计** (logical design) 阶段, 我们将**实体-联系模型**定义的概念模式映射到关系模式. 在**物理设计** (physical design) 阶段, 我们将具体设计数据库的物理特性, 例如文件组织形式和索引结构.

3.1.1 定义

实体-联系模型 (entity-relationship model) 简称 E-R 模型. 顾名思义, 它由**实体**和**联系**构成.

- **实体** (entity) 是可区别于其他对象的对象, 例如一个大学教师. 每个实体都有一组属性, 并且某个属性子集 (即候选键) 的值能够唯一地标识对应的实体. **实体集** (entity set) 是同类型实体的集合, 例如一所大学的所有教师的集合可定义为 instructor 实体集. 实体集在 E-R 图中用矩形表示, 包括实体集的名称和所有属性的名称. 实体集的实例称为**外延** (extension), 类似于关系与关系实例之间的区别.

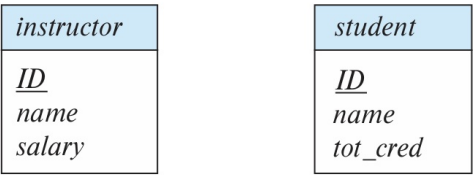


Figure 6.1 E-R diagram showing entity sets *instructor* and *student*.

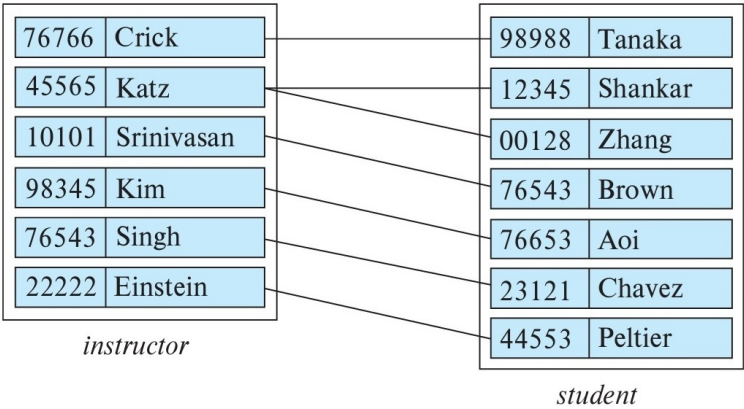


Figure 6.2 Relationship set *advisor* (only some attributes of *instructor* and *student* are shown).

- **联系** (relationship) 是实体间的关联. **联系集** (relationship set) 是同类型联系的集合. 联系集在 E-R 图中用菱形表示, 通过线条连接到不同实体集:

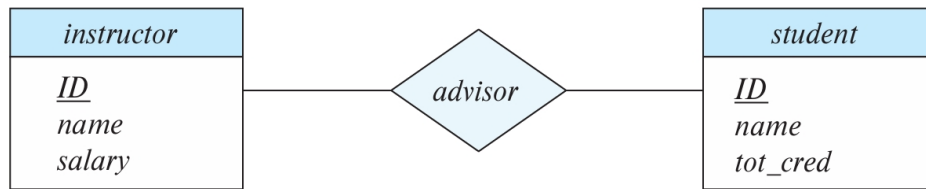


Figure 6.3 E-R diagram showing relationship set *advisor*.

实体在联系中扮演的功能称为实体的**角色** (role)

同一实体集能以不同角色多次**参与** (participate) 一个联系集 (称为递归的联系集):

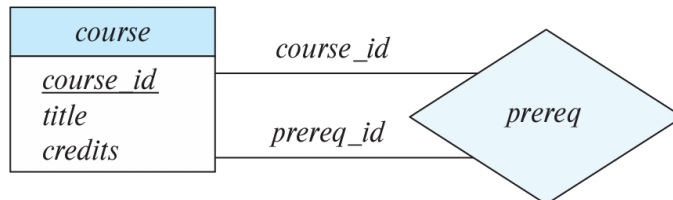


Figure 6.4 E-R diagram with role indicators.

联系也以具有**描述性属性** (descriptive attribute):

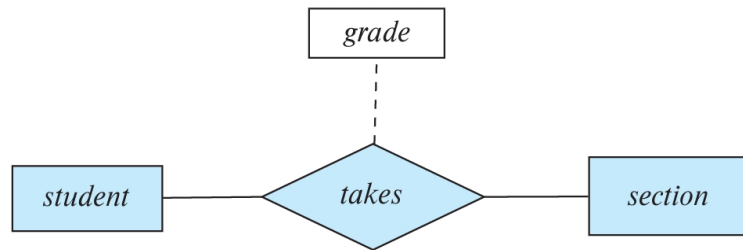


Figure 6.5 E-R diagram with an attribute attached to a relationship set.

非二元联系也可用 E-R 图表示, 下面是一个三元联系集 *proj_guide* 的示例:

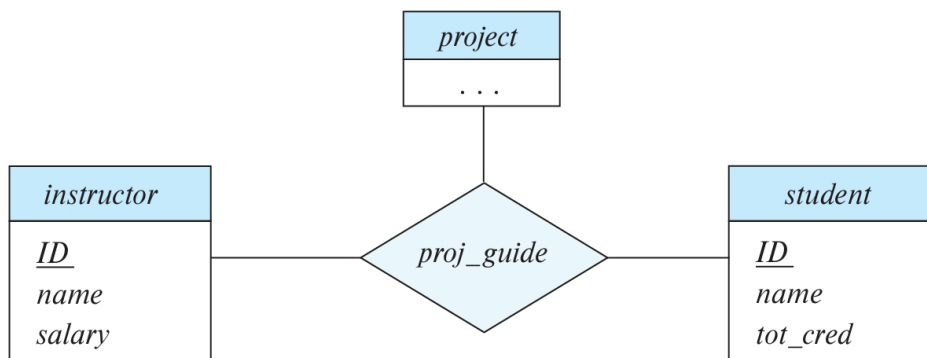


Figure 6.6 E-R diagram with a ternary relationship *proj_guide*.

3.1.2 属性

每个属性都有一个可取值的集合, 称为该属性的**域** (domain)

例如 *course_id* 属性的域是特定长度以内的所有文本字符串的集合.

- **简单属性 (simple attribute)** 又称原子属性, 即不可再分的属性.
- **复合属性 (composite attribute)** 是可以划分出子属性 (又称成员属性) 的属性. 它们可以将相关属性整合起来, 使模型更加清晰:

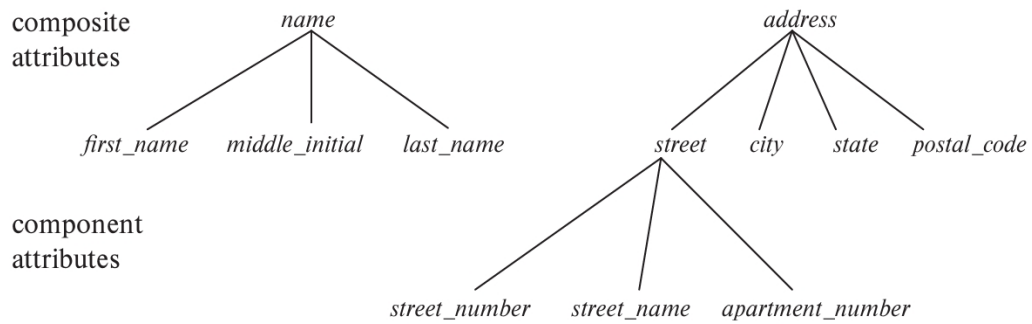


Figure 6.7 Composite attributes instructor *name* and *address*.

- **单值属性 (single-valued attribute)** 对于一个特定的实体只有单独的一个值。
多值属性 (multivalued attribute) 对于一个特定的实体可能有一组值。
 例如我们可以为 instructor 表添加一个 phone_number 属性，理论上它可以有任意自然数个数值。
- **派生属性 (derived attribute)** 的值可由其他相关属性和相关联的实体导出。
 例如我们可以为 instructor 表添加一个 students_advised 属性，表示一位教师指导了多少名学生，它的值可通过统计与一位教师相关联的所有 student 实体的数目来导出。
 我们还可以为 instructor 表添加一个 age 属性，它可由 date_of_birth 和当前日期导出。
 此时我们称 date_of_birth 为基属性，age 为派生属性，它的值并不存储，只在需要时计算出来。

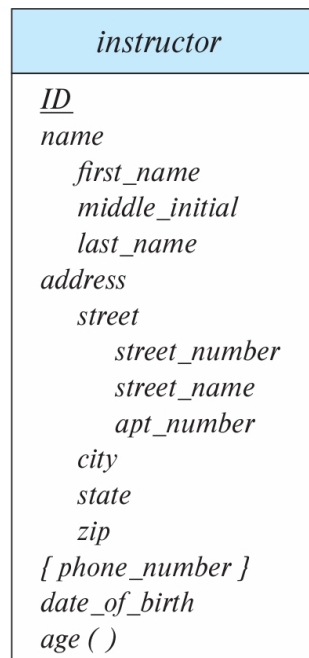


Figure 6.8 E-R diagram with composite, multivalued, and derived attributes.

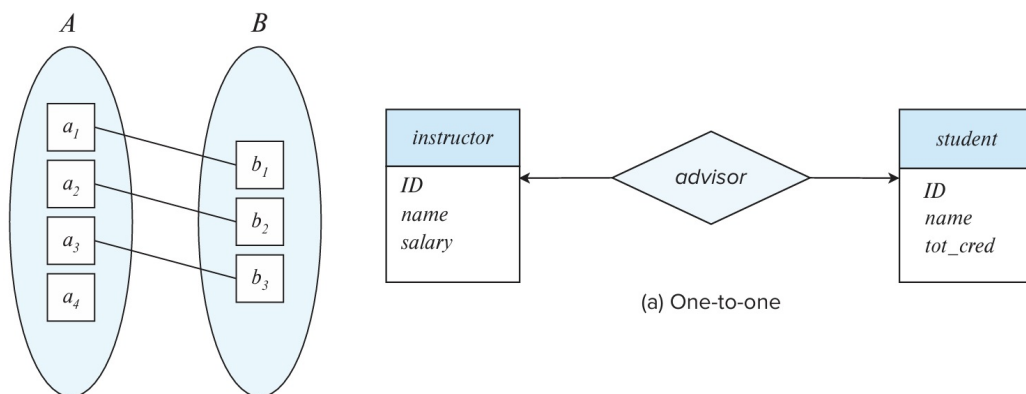
3.1.3 映射基数

映射基数 (mapping cardinality) 表示一个实体能通过一个联系集关联的另一些实体的数量。

对于实体集 A , B 之间的二元联系集 R 来说，其映射基数有以下可能：

(在 E-R 图中使用有向线段和无向线段进行区分)

- **一对一 (ont-to-one)**
 A 中的一个实体通过联系 R 至多与 B 中一个实体相关联，反之亦然。



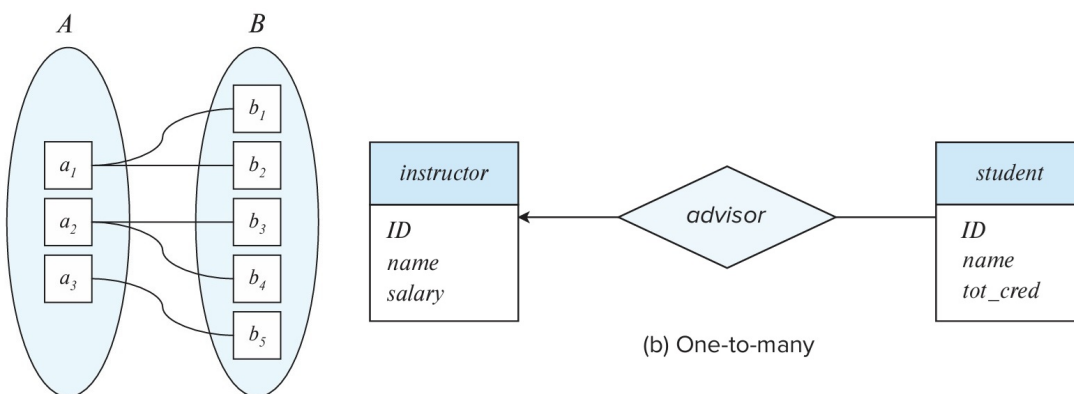
(a) One-to-one

在一对一的二元联系中，参与实体集的主键都可构成联系集的主键，唯一地标识联系集中的联系。

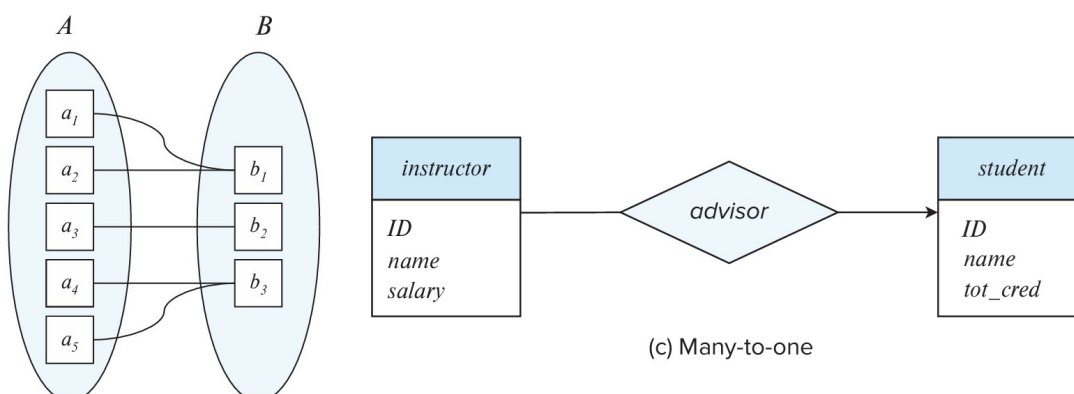
- **一对多 (one-to-many)**

A 中的一个实体通过联系 R 可与 B 中任意数量的实体相关联，但 B 中的一个实体通过联系 R 至多与 A 中一个实体相关联。

多对一 (many-to-one) 就是上述情况的反面。



(b) One-to-many

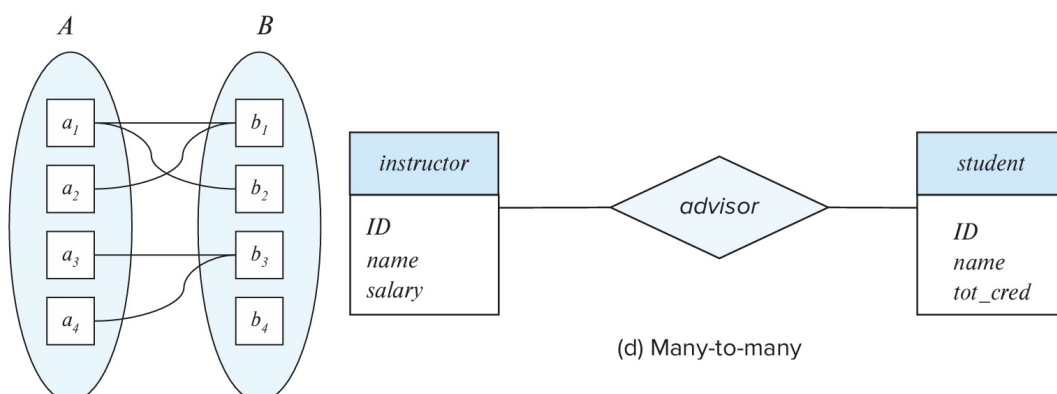


(c) Many-to-one

在一对多或多对一的二元联系中，参与实体集中“多”的一方的主键可构成联系集的主键。

- **多对多 (many-to-many)**

A 中的一个实体通过联系 R 可与 B 中任意数量的实体相关联，反之亦然。



(d) Many-to-many

在多对多的二元联系中，参与实体集的主键的并集 $\text{primary-key}(A)$ 和 $\text{primary-key}(B)$ 可构成联系集的主键。

- 此外，我们使用双线表示一个实体集在联系集中的全部参与。

例如下图中的双线表明学生必须拥有至少一名导师：

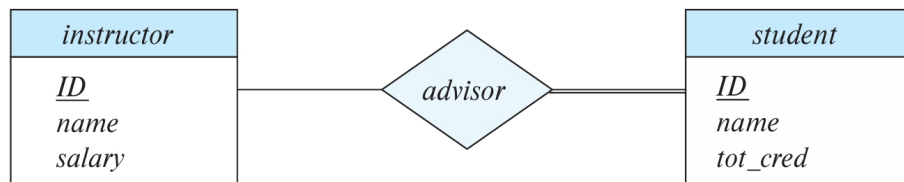


Figure 6.12 E-R diagram showing total participation.

- E-R 图还可以描述更复杂的约束，限制每个实体参与联系的次数。
其格式为 $l..h$ ，其中 l 代表最小基数， h 代表最大基数。
例如 $1..1$ 代表必须参与且仅能参与一次，而 $0..*$ 代表可以参与任意自然数次：

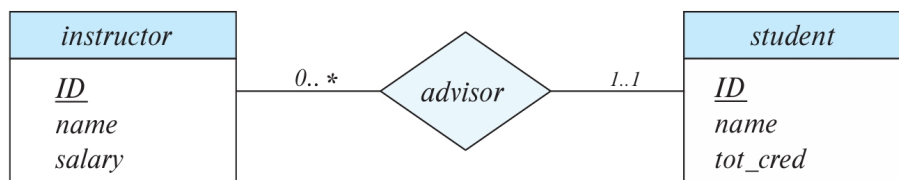


Figure 6.13 Cardinality limits on relationship sets.

对于涉及实体集 $E_1, E_2, \dots, E_n$ 的非二元联系集 R ，假设其具有描述性属性 $\{a_1, \dots, a_m\}$
若该联系没有基数约束，则联系集 R 的主键可以设为：

$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n) \cup \{a_1, \dots, a_m\}$$

若有基数约束，则主键的选择就要复杂多了。

一般情况下，我们要求一个非二元联系集至多有一个箭头指出。

这是因为有多个箭头指出的非二元联系集的解释有歧义。

假设实体集 E_1, E_2, E_3, E_4 之间有联系集 R ，且有两个箭头分别指向 E_3, E_4 ，则它有两种可能的解释：

- 解释一：**
来自 E_1, E_2 的一个特定实体组合至多和一个来自 E_3, E_4 的实体组合相关联。
此时联系集 R 的主键可用 E_1, E_2 的主键的并集 $\text{primary-key}(E_1) \cup \text{primary-key}(E_2)$ 构造。
- 解释二：**
来自 E_1, E_2, E_3 的一个特定实体集合至多和一个来自 E_4 的实体相关联。
同时，来自 E_1, E_2, E_4 的一个特定实体集合至多和一个来自 E_3 的实体相关联。
此时联系集 R 的主键可用 E_1, E_2, E_3 的主键的并集 $\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \text{primary-key}(E_3)$ 构造。
或者可用 E_1, E_2, E_4 的主键的并集 $\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \text{primary-key}(E_4)$ 构造。

后面我们会使用**函数依赖** (functional dependency) 来明确地指定使用上述两种解释的哪一种。

3.1.4 弱实体集

弱实体集 (weak entity set) 中实体的唯一标识需要依赖其他实体集 (称为标识性实体集)

非弱实体集的实体集被称为**强实体集** (strong entity set)

每个弱实体必须和一个标识性实体相关联。

换言之，弱实体集被称为**存在依赖** (existence dependent) 于标识性实体集。

标识性实体集被称为**拥有** (own) 它所标识的弱实体集。

将弱实体集与标识性实体集相关联的联系被称为**标识性联系** (identifying relationship)

它是从弱实体集到标识性实体集的多对一联系，并且弱实体集在联系中的参与是全部的。

标识性联系集不应该有任何描述性属性，因为任何这样的属性都可以与弱实体集相关联。

假定我们将 $\text{section}(\text{course_id}, \text{sec_id}, \text{semester}, \text{year})$ 中的 course_id 删除，

则 section 实体集就没有足够的属性来唯一地标识一个特定的 section 实体。

此时它需要通过标识性联系 sec_course 依赖于 course 强实体集。

于是它的主键由 course 的主键 course_id 加上弱实体集 section 的分辨符 $\{\text{course_id}, \text{semester}, \text{year}\}$ 构成。

在 E-R 图中，我们通过双边框的矩形描述弱实体集，其分辨符被加上虚的下划线。

关联弱实体集和标识性强实体集的联系集以双边框的菱形表示。



Figure 6.14 E-R diagram with a weak entity set.

图中的双线表示弱实体集 section 在 sec_course 联系中的参与是全部的, 这意味着每个课程段必须通过 sec_course 与某门课程相关联. 而从 sec_course 指向 course 的箭头表示每个课程段与单门课程相关联.

3.1.5 E-R 图的记号

下图总结了我们在 E-R 图表示法中用到的符号集:

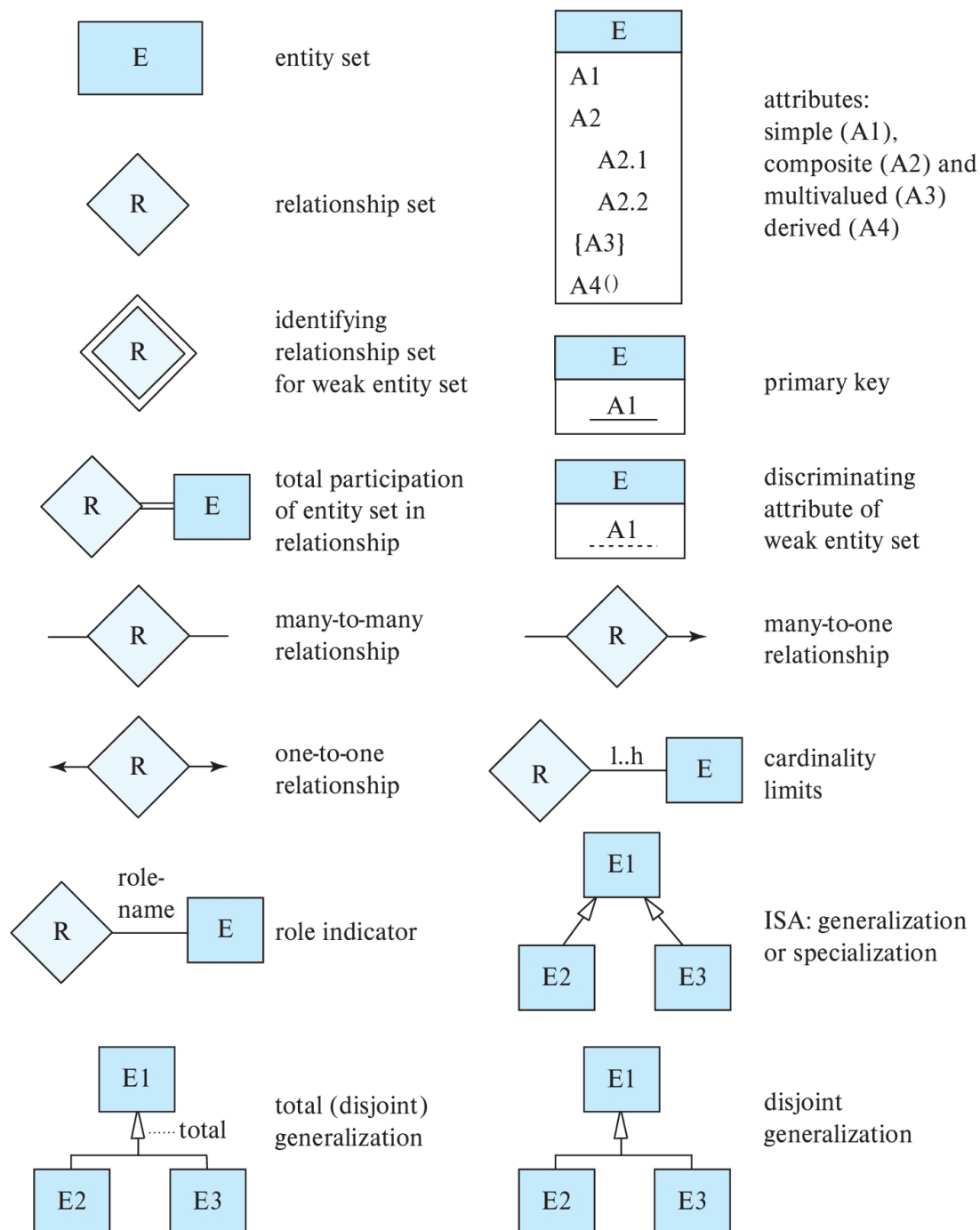
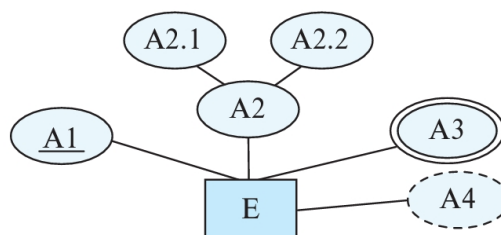


Figure 6.26 Symbols used in the E-R notation.

实体属性的表示方法如下: (联系的属性可以类似地表示)

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



联系上的基数约束的表示方法不唯一，下图给出了一些其他的记号:

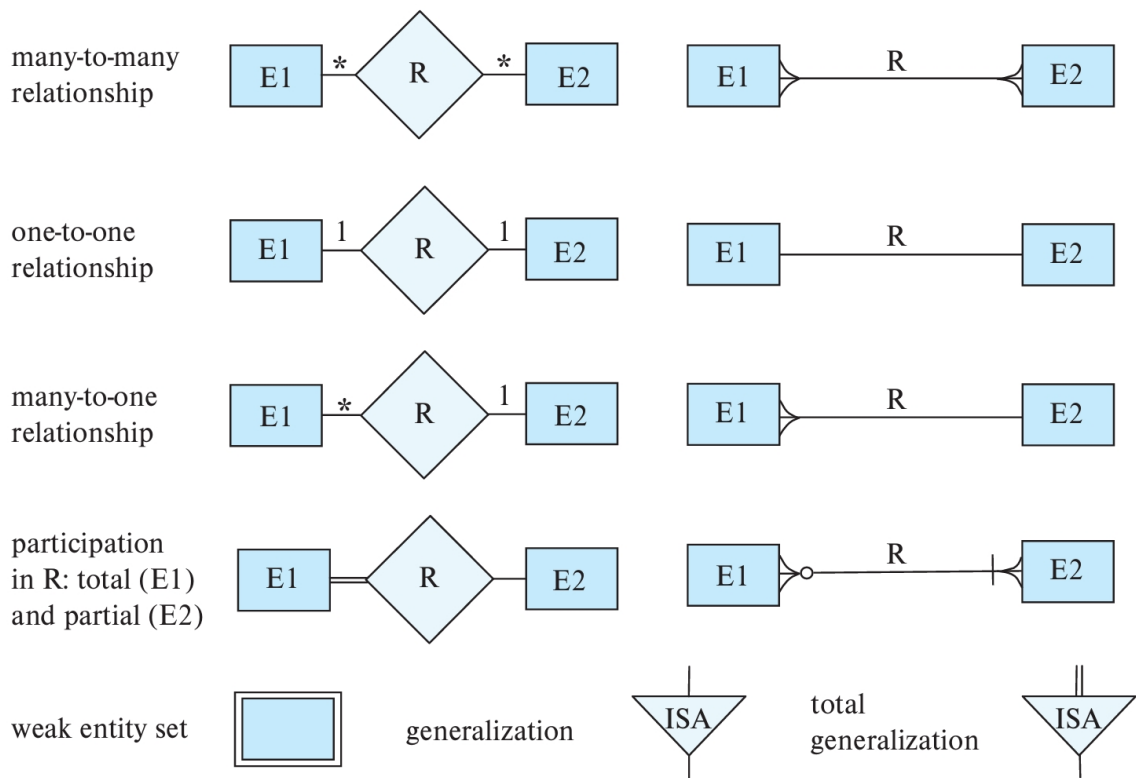


Figure 6.27 Alternative E-R notations.

3.1.6 从 E-R 图生成关系模式

(待补充)

3.1.7 示例

好的 E-R 设计不包含冗余的属性.

以大学数据库为例:

- 其实体集如下:
 - classroom*: with attributes (building, room_number, capacity).
 - department*: with attributes (dept_name, building, budget).
 - course*: with attributes (course_id, title, credits).
 - instructor*: with attributes (ID, name, salary).
 - section*: with attributes (course_id, sec_id, semester, year).
 - student*: with attributes (ID, name, tot_cred).
 - time_slot*: with attributes (time_slot_id, {(day, start_time, end_time)}).

- 其联系集如下:
 - *inst_dept*: relating instructors with departments.
 - *stud_dept*: relating students with departments.
 - *teaches*: relating instructors with sections.
 - *takes*: relating students with sections, with a descriptive attribute *grade*.
 - *course_dept*: relating courses with departments.
 - *sec_course*: relating sections with courses.
 - *sec_class*: relating sections with classrooms.
 - *sec_time_slot*: relating sections with time slots.
 - *advisor*: relating students with instructors.
 - *prereq*: relating courses with prerequisite courses.

可以验证没有任何一个实体集包含因某个联系集而造成冗余的任何属性。
其 E-R 图如下:

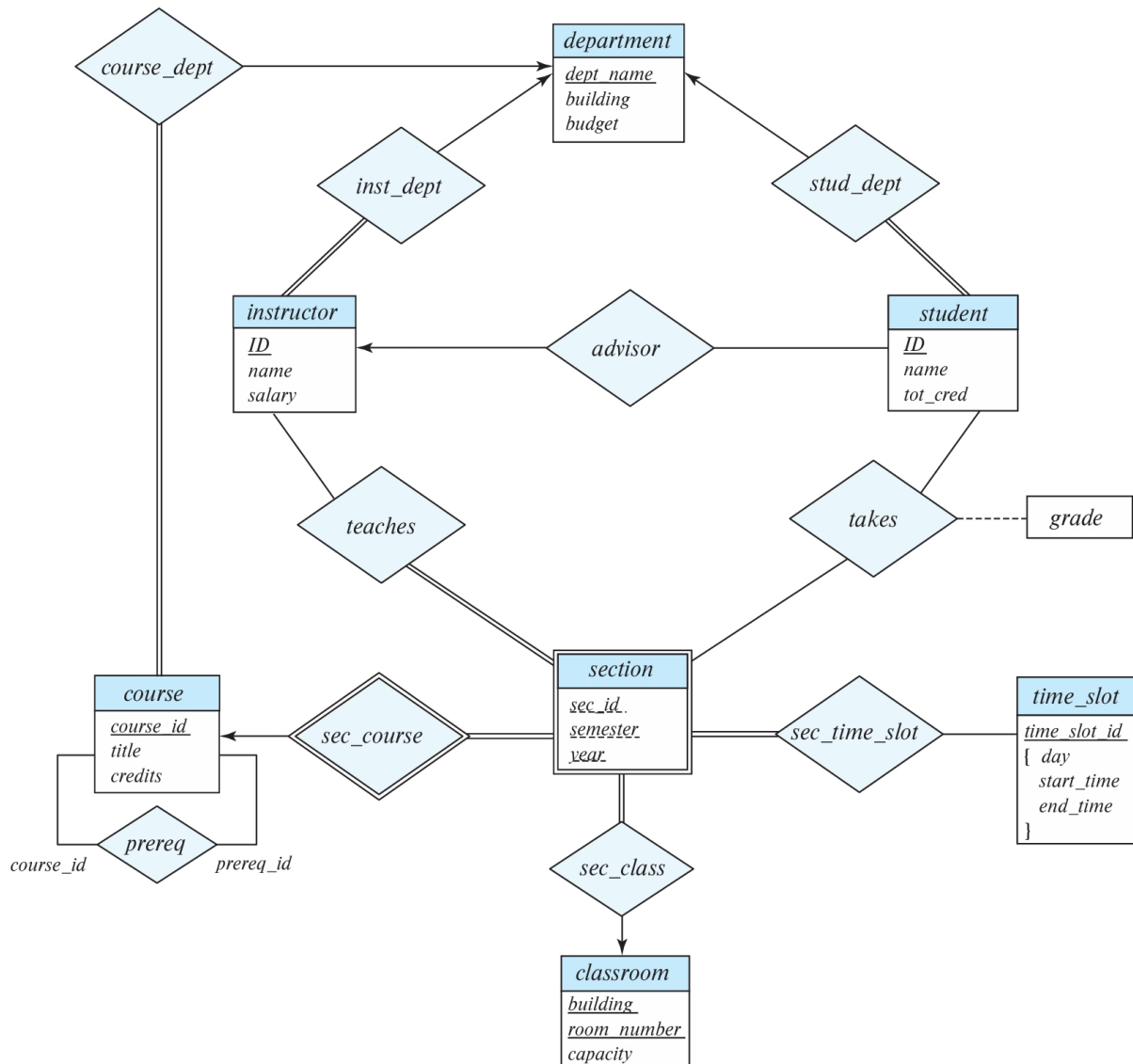


Figure 6.15 E-R diagram for a university enterprise.

从上述 E-R 图导出的关系模式如下:

```
classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, credits)
instructor(ID, name, salary)
student(ID, name, tot_cred)
time_slot (time_slot_id, day, start_time, end_time)
section (course_id, sec_id, semester, year)
```

Figure 6.16 Schemas derived from the entity sets in the E-R diagram in Figure 6.15.

```
teaches (ID, course_id, sec_id, semester, year)
takes (ID, course_id, sec_id, semester, year, grade)
prereq (course_id, prereq_id)
advisor (s_ID, i_ID)
sec_course (course_id, sec_id, semester, year)
sec_time_slot (course_id, sec_id, semester, year, time_slot_id)
sec_class (course_id, sec_id, semester, year, building, room_number)
inst_dept (ID, dept_name)
stud_dept (ID, dept_name)
course_dept (course_id, dept_name)
```

Figure 6.17 Schemas derived from relationship sets in the E-R diagram in Figure 6.15.

3.1.8 什么是好的关系设计

从上述示例我们可以直接从 E-R 模型来生成一组关系模式。
所生成的关系模式集的好坏取决于 E-R 模型的质量。

```
classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)
```

Figure 7.1 Database schema for the university example.

假设我们需要设计 inst_dept 模式:

inst_dept(ID, name, salary, dept_name, building, budget)

它是 instructor 和 department 关系之间自然连接的结果，显然不是什么好的设计。

在使用 instructor 和 department 的原始设计中，我们对每个预算 budget 的数额只存储一次。

而引入 inst_dept 关系会重复存储预算 budget 的数额，并且增加了更新操作的代价。

即使我们决定容忍冗余的问题，inst_dept 关系仍然存在其他问题。

假设我们在大学里创立一个新的系，在新的设计中，

我们无法直接表达关于一个系的信息 (dept_name, building, budget)，除非该系在大学中至少有一位教师。

这是因为 inst_dept 关系中的元组需要 ID、name 和 salary 的值。

这意味着 inst_dept 关系不能记录新成立的系的相关信息，直到这个新系录用了第一位教师为止。

避免 inst_dept 关系模式中的信息冗余问题的唯一方式是将其分解为两个小的关系模式 (即 instructor 和 department)。

inst_dept(ID, name, salary, dept_name, building, budget)

⇓

instructor(ID, name, dept_name, salary)

department(dept_name, building, budget)

但并非所有的模式分解都是有益的，一个简单的例子如下：

employee(ID, name, street, city, salary)

⇓

employee1(ID, name)

employee2(name, street, city, salary)

name 属性的重名问题会导致信息丢失：

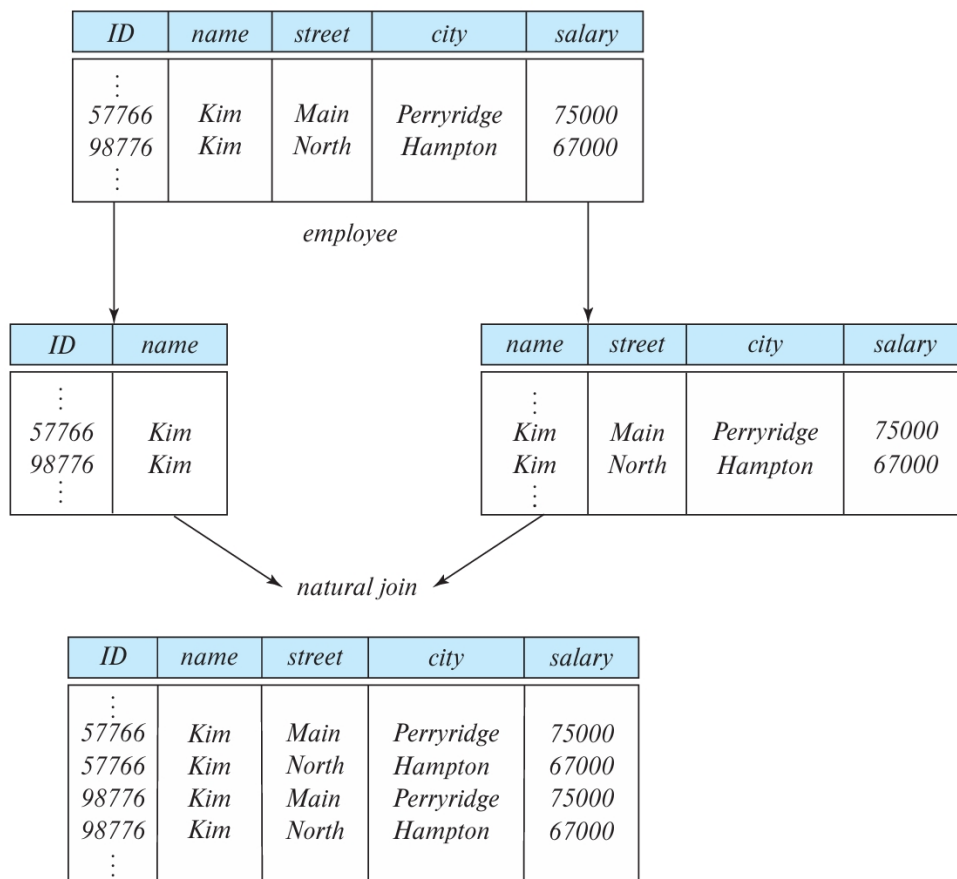


Figure 7.3 Loss of information via a bad decomposition.

我们将 *employee* 模式分解后，尝试通过自然连接重新生成原始元组，却发现额外生成了两个新的元组。

这两个新的元组将属于这两个名为 Kim 的职员的数据值错误地混合在一起。

虽然我们拥有更多的元组，但实际上我们所拥有的信息却更少了：

我们能够指出一个特定的 street、city 和 salary 属于某个名为 Kim 的人，但是我们无法区分出是哪一个 Kim。

我们有更多的元组，却更少的信息，这似乎有悖常理，但事实确实如此。

分解后的两个模式无法表示 name 与 street、city 和 salary 之间的联系，而这个联系正是缺失的信息。

这样有信息丢失的分解称为**有损分解** (lossy decomposition)

那些没有信息丢失的称为**无损分解** (lossless decomposition)

具体来说，假设属性集 R_1 和 R_2 构成 R 的分解 (即 $R = R_1 \cup R_2$)

考虑关系模式 $r(R)$ 的任意实例 r

若投影结果的自然连接是原始关系的真超集 (即 $r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$)，则 $R = R_1 \cup R_2$ 为有损分解。

若投影结果的自然连接恰好是原始关系 (即 $r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$)，则 $R = R_1 \cup R_2$ 为无损分解。

用于设计关系数据库的方法是使用一个**规范化** (normalization) 的过程。

其目标是生成一组关系模式，允许我们存储信息并避免不必要的冗余，同时还允许我们轻松地检索信息。

如果一个给定的关系模式不是“良构的” (即它受信息冗余的影响)，

那么我们将它无损分解成许多较小的关系模式，使得每个模式都满足适当的**范式** (normal forms)

我们通常使用**函数依赖** (functional dependency) 来确定一个关系模式是否符合某种理想的范式。

3.2 函数依赖

3.2.1 定义

关系模式的约束可以被表示为键 (超键、候选键、主键和外键)，或表示为下面引入的函数依赖。

超键是能够唯一标识整个元组的属性集，函数依赖让我们可以表达唯一标识特定属性的值的约束。

在对函数依赖的讨论中，符号 “=” 代表数学意义上的相等，而不是 SQL 的三值逻辑含义。
换言之，在函数依赖性的讨论中，我们假设没有空值。

考虑关系模式 $r(R)$ ，设 α, β 是属性集 R 的子集 (记为 $\alpha, \beta \subseteq R$)

- 给定关系模式 $r(R)$ 的实例 r
若对于关系实例 r 中的任意元组 t_1, t_2 ，只要 $t_1[\alpha] = t_2[\alpha]$ 就有 $t_1[\beta] = t_2[\beta]$ ，
则我们称关系实例 r 满足函数依赖 $\alpha \mapsto \beta$
(如果 $\beta \subseteq \alpha$ ，则函数依赖 $\alpha \mapsto \beta$ 必定被所有关系实例满足，因此我们称其为平凡的函数依赖)
- 若关系模式 $r(R)$ 的任意合法实例 r 都满足函数依赖 $\alpha \mapsto \beta$ ，
则我们称函数依赖 $\alpha \mapsto \beta$ 在关系模式 $r(R)$ 上成立。

- A functional dependency is **trivial(平凡的)** if it is satisfied by all instances of a relation, e.g.,

$customer_name, loan_number \rightarrow customer_name$

$customer_name \rightarrow customer_name$

- In general, $\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$

- Full dependency and partially dependency

- β is **fully dependent** on α , if there is no proper subset α' of α such that $\alpha' \rightarrow \beta$. Otherwise, β is **partially dependent** on α

以 3.1.8 节中的 inst_dept 模式为例，其无损分解为：

inst_dept(ID, name, salary, dept_name, building, budget)

⇓

instructor(ID, name, dept_name, salary)

department(dept_name, building, budget)

- 在该模式中函数依赖 $dept_name \mapsto budget$ 成立，
因为每个系 (由 dept_name 唯一标识) 都存在唯一的预算数额 budget。
- 属性集 (ID, dept_name) 构成 inst_dept 模式的一个超键，我们可将这一事实写作：

$(ID, dept_name) \mapsto (name, salary, building, budget)$

我们将以两种方式来使用函数依赖:

- ① 测试关系的实例, 看它们是否满足一个给定的函数依赖集 $\mathcal{F}$
- ② 声明合法关系集上的约束.

因此我们将只关注满足给定函数依赖集的那些关系实例.

如果我们希望把注意力放到模式 $r(R)$ 上满足函数依赖集 $\mathcal{F}$ 的关系, 则我们要求 $\mathcal{F}$ 在 $r(R)$ 上成立.

函数依赖是单向的, 例如下图中 $A \mapsto C$ 的函数依赖成立, 但 $C \mapsto A$ 的函数依赖不成立:

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4

Figure 7.4 Sample instance of relation r .

值得注意的是, 一个关系实例可能满足的某些函数依赖并不需要在整个关系模式上成立.

例如 classroom 的某个关系实例 (如下图所示) 满足 $room_number \mapsto capacity$ 的函数依赖,

但我们相信这个函数依赖并不在整个 classroom 关系模式上成立,

因为在现实世界中, 不同教学楼里的两个教室可以具有相同的房间号, 但却具有不同的空间容量.

$building$	$room_number$	$capacity$
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Figure 7.5 An instance of the *classroom* relation.

3.2.2 无损分解

回顾无损分解的定义:

假设属性集 R_1 和 R_2 构成 R 的分解 (即 $R = R_1 \cup R_2$)

考虑关系模式 $r(R)$ 的任意实例 r

若投影结果的自然连接是原始关系的真超集 (即 $r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$), 则 $R = R_1 \cup R_2$ 为有损分解.

若投影结果的自然连接恰好是原始关系 (即 $r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$), 则 $R = R_1 \cup R_2$ 为无损分解.

我们可以用函数依赖来说明什么时候特定的分解是无损的.

$R = R_1 \cup R_2$ 构成 R 的一个无损分解的条件是:

"函数依赖 $R_1 \cap R_2 \mapsto R_1$ 和 $R_1 \cap R_2 \mapsto R_2$ 中至少有一个在关系模式 $r(R)$ 上成立".

换言之, 如果 $R_1 \cap R_2$ 要么构成 R_1 的超键要么构成 R_2 的超键, 则 $R = R_1 \cup R_2$ 就是一个无损分解.

假设我们将一个关系模式 $r(R)$ 分解为 $r_1(R_1)$ 和 $r_2(R_2)$, 其中 $R_1 \cap R_2 \mapsto R_1$,

那么必须对分解后的模式施加以下 SQL 约束以确保它们的内容与原始模式保持一致:

- ① $R_1 \cap R_2$ 是 $r_1(R_1)$ 的主键
(此约束强制实施该函数依赖)
- ② $R_1 \cap R_2$ 是从 $r_2(R_2)$ 引用 r_1 的外键
(这个约束确保 r_2 中的每个元组在 r_1 中都有一个匹配元组)

如果进一步分解 $r_1(R_1)$ 或 $r_2(R_2)$, 只要分解确保属性集 $R_1 \cap R_2$ 都在一个关系中, 那么 $r_1(R_1)$ 或 $r_2(R_2)$ 上的主键或外键约束将由该关系来继承.

3.2.3 Armstrong 公理

给定在关系模式 $r(R)$ 上成立的一个函数依赖集 $\mathcal{F}$, 我们可以推出其他函数依赖.

例如给定模式 $r(A, B, C)$

若函数依赖 $A \mapsto B$ 和 $B \mapsto C$ 在关系实例 r 上成立, 我们可以推断函数依赖 $A \mapsto C$ 也一定在 r 上成立.

我们称能从函数依赖集 $\mathcal{F}$ 推出的所有函数依赖的集合为 $\mathcal{F}$ 的**闭包** (closure), 记为 $\text{cl}(\mathcal{F})$

给定关系模式 $r(R)$

如果每一个满足函数依赖集 $\mathcal{F}$ 的实例也满足函数依赖 f ,

则我们称 $r(R)$ 上的函数依赖 f 被 $r(R)$ 上的函数依赖集 $\mathcal{F}$ 所**逻辑蕴含** (logically imply)

例如给定关系模式 $r(A, B, C, G, H, I)$ 和函数依赖集 $\mathcal{F}$:

$$\begin{aligned} A &\mapsto B \\ A &\mapsto C \\ B &\mapsto H \\ (C, G) &\mapsto H \\ (C, G) &\mapsto I \end{aligned}$$

可以看出函数依赖 $A \mapsto H$ 是被逻辑蕴含的.

(Armstrong 公理)

我们可以通过反复应用以下三条规则去寻找被逻辑蕴含的函数依赖, 即计算函数依赖集 $\mathcal{F}$ 的闭包 $\text{cl}(\mathcal{F})$.

- ① **自反律 (reflexivity rule):**
若属性集 $\beta \subseteq \alpha \subseteq R$, 则函数依赖 $\alpha \mapsto \beta$ 在关系模式 $r(R)$ 上一定成立.
- ② **增广律 (augmentation rule):**
若函数依赖 $\alpha \mapsto \beta$ 在关系模式 $r(R)$ 上成立,
则任意给定属性集 $\gamma \subseteq R$, 函数依赖 $(\alpha \cup \gamma) \mapsto (\beta \cup \gamma)$ 在关系模式 $r(R)$ 上一定成立.
- ③ **传递律 (transitivity rule):**
若函数依赖 $\alpha \mapsto \beta$ 和 $\beta \mapsto \gamma$ 在关系模式 $r(R)$ 上成立,
则函数依赖 $\alpha \mapsto \gamma$ 在关系模式 $r(R)$ 上也成立.

虽然 Armstrong 公理是完备的, 但直接用它们来计算闭包 $\text{cl}(\mathcal{F})$ 会很麻烦.

为进一步简化这个问题, 我们列出附加的规则:

- ④ **复合律 (composition rule):**
函数依赖 $\alpha \mapsto \beta$ 和 $\alpha \mapsto \gamma$ 在关系模式 $r(R)$ 上成立,
等价于函数依赖 $\alpha \mapsto (\beta \cup \gamma)$ 在关系模式 $r(R)$ 上成立.
- ⑤ **伪传递律 (pseudo-transitivity rule):**
若函数依赖 $\alpha \mapsto \beta$ 和 $(\beta \cup \gamma) \mapsto \eta$ 在关系模式 $r(R)$ 上成立,
则函数依赖 $(\alpha \cup \gamma) \mapsto \eta$ 在关系模式 $r(R)$ 上也成立.

3.2.4 属性闭包算法

给定属性集 α, β

若 $\alpha \mapsto \beta$, 则我们称属性集 β 被 α **函数决定** (functionally determine)

要测试一个属性集 α 是否为超键, 我们必须设计一个算法, 用于计算被 α 函数决定的属性集.

一种方式是计算函数依赖集 $\mathcal{F}$ 的闭包 $\text{cl}(\mathcal{F})$,

找出所有左侧为 α 的函数依赖, 然后合并所有这些依赖的右侧.

但这么做是有代价的, 因为 $\text{cl}(\mathcal{F})$ 可能很大:

```

 $F^+ = F$ 
apply the reflexivity rule /* Generates all trivial dependencies */
repeat
  for each functional dependency  $f$  in  $F^+$ 
    apply the augmentation rule on  $f$ 
    add the resulting functional dependencies to  $F^+$ 
  for each pair of functional dependencies  $f_1$  and  $f_2$  in  $F^+$ 
    if  $f_1$  and  $f_2$  can be combined using transitivity
      add the resulting functional dependency to  $F^+$ 
until  $F^+$  does not change any further

```

Figure 7.7 A procedure to compute F^+ .

另一种用于计算被 α 函数决定的属性集的高效算法称为**属性闭包算法**.
 记函数依赖集 $\mathcal{F}$ 下被 α 函数决定的所有属性的集合称为 $\mathcal{F}$ 下 α 的闭包, 记为 $\text{cl}(\alpha)$
 最坏情况下, 其时间复杂度是 $O(|\mathcal{F}|^2)$

```

result :=  $\alpha$ ;
repeat
  for each functional dependency  $\beta \rightarrow \gamma$  in  $F$  do
    begin
      if  $\beta \subseteq \text{result}$  then result := result  $\cup \gamma$ ;
    end
until (result does not change)

```

Figure 7.8 An algorithm to compute α^+ , the closure of α under F .

例如给定关系模式 $r(A, B, C, G, H, I)$ 和函数依赖集 $\mathcal{F}$:

$$\begin{aligned}
 A &\mapsto B \\
 A &\mapsto C \\
 B &\mapsto H \\
 (C, G) &\mapsto H \\
 (C, G) &\mapsto I
 \end{aligned}$$

考虑使用属性闭包算法来计算属性集 $\alpha = (A, G)$ 的闭包 $\text{cl}(A, G)$

从 $\text{result} = \alpha = (A, G)$ 开始, 第一个循环如下:

- ① 注意到 $A \mapsto B$ 和 $A \subseteq \text{result} = (A, G)$, 故 $\text{result} \leftarrow \text{result} \cup B = (A, B, G)$
- ② 注意到 $A \mapsto C$ 和 $A \subseteq \text{result} = (A, B, G)$, 故 $\text{result} \leftarrow \text{result} \cup C = (A, B, C, G)$
- ③ 注意到 $B \mapsto H$ 和 $B \subseteq \text{result} = (A, B, C, G)$, 故 $\text{result} \leftarrow \text{result} \cup H = (A, B, C, G, H)$
- ④ 注意到 $(C, G) \mapsto H$ 和 $(C, G) \subseteq \text{result} = (A, B, C, G, H)$, 故 $\text{result} \leftarrow \text{result} \cup H = (A, B, C, G, H)$
- ⑤ 注意到 $(C, G) \mapsto I$ 和 $(C, G) \subseteq \text{result} = (A, B, C, G, H)$, 故 $\text{result} \leftarrow \text{result} \cup I = (A, B, C, G, H, I)$

第二次循环不会有新属性加入 result,

故算法终止, 得到 $\text{cl}(\alpha) = \text{result} = \text{cl}(A, G) = (A, B, C, G, H, I)$

属性闭包算法的用途:

- ① 测试属性集 α 是否为超键, 即计算闭包 $\text{cl}(\alpha)$ 并检查 $\text{cl}(\alpha)$ 是否等于属性集 R .
- ② 检查函数依赖 $\alpha \mapsto \beta$ 是否成立, 即计算闭包 $\text{cl}(\alpha)$ 并检查 $\text{cl}(\alpha)$ 是否包含属性集 β .

- ③ 计算函数依赖集 $\mathcal{F}$ 的闭包 $\text{cl}(\mathcal{F})$:
对于任意 $\alpha \subseteq \mathbb{R}$, 我们计算其闭包 $\text{cl}(\alpha)$, 构造对应的函数依赖 $\alpha \mapsto \beta$ ($\forall \beta \subseteq \text{cl}(\alpha)$)
最终汇总得到函数依赖集 $\mathcal{F}$ 的闭包 $\text{cl}(\mathcal{F})$.

3.2.5 正则覆盖

如果我们可以去除函数依赖的一个属性而不改变该函数依赖集的闭包, 则称该属性是**无关的** (extraneous)
给定函数依赖集 $\mathcal{F}$, 考虑其中的函数依赖 $\alpha \mapsto \beta$:

- **从左侧移除无关属性:**
若 $A \in \alpha$ 且 $\mathcal{F}$ 逻辑蕴含 $(\mathcal{F} - \{\alpha \mapsto \beta\}) \cup \{(\alpha - A) \mapsto \beta\}$, 则属性 A 在 α 中是无关的.
- **从右侧移除无关属性:**
若 $B \in \beta$ 且 $\mathcal{F}$ 逻辑蕴含 $(\mathcal{F} - \{\alpha \mapsto \beta\}) \cup \{\alpha \mapsto (\beta - B)\}$, 则属性 B 在 β 中是无关的.

下面介绍如何有效地检验一个属性是否是无关的.

令 $r(R)$ 为一个关系模式, 且 $\mathcal{F}$ 是在 R 上成立的给定函数依赖集.

考虑函数依赖 $\alpha \mapsto \beta$:

- 若要检验 $A \in \alpha$ 是否是无关属性, 则可在 $\mathcal{F}$ 下计算 $\text{cl}(\alpha - A)$, 判断其是否包含 β 的所有属性.
- 若要检验 $B \in \beta$ 是否是无关属性,
则可在 $(\mathcal{F} - \{\alpha \mapsto \beta\}) \cup \{\alpha \mapsto (\beta - B)\}$ 计算 $\text{cl}(\alpha)$, 判断其是否属性 B .

如果更新违反了 $\mathcal{F}$ (或者说 $\text{cl}(\mathcal{F})$) 中的任意函数依赖, 则系统必须回滚该更新.

我们可以通过测试与给定函数依赖集具有相同闭包的一个简化集的方式来减小花费在违反检测方面的开销.

$\mathcal{F}$ 的**正则覆盖** (canonical cover) $\mathcal{F}_c$ 就是这样一个简化集, 它具备以下性质:

- ① $\mathcal{F}_c$ 中任何函数依赖都不包含无关属性
- ② $\mathcal{F}_c$ 中每个函数依赖的左侧都是唯一的,
即在 $\mathcal{F}_c$ 中不存在两个依赖 $\alpha_1 \mapsto \beta_1$ 和 $\alpha_2 \mapsto \beta_2$ 满足 $\alpha_1 = \alpha_2$

$F_c = F$

repeat

Use the union rule to replace any dependencies in F_c of the form

$\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$.

Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous attribute either in α or in β .

/* Note: the test for extraneous attributes is done using F_c , not F^* */

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$ in F_c .

until (F_c does not change)

Figure 7.9 Computing canonical cover.

例如给定关系模式 $r(A, B, C)$ 和函数依赖集 $\mathcal{F}$:

$A \mapsto (B, C)$

$B \mapsto C$

$A \mapsto B$

$(A, B) \mapsto C$

考虑使用正则覆盖算法来计算函数依赖集 $\mathcal{F}$ 的正则覆盖 $\mathcal{F}_c$.

从 $\mathcal{F}_c = \mathcal{F}$ 开始, 第一个循环如下:

- ① 注意到 $A \mapsto (B, C)$ 和 $A \mapsto B$ 有相同的左侧,
于是 $\mathcal{F}_c = \{A \mapsto (B, C), B \mapsto C, (A, B) \mapsto C\}$
- ② 根据 $B \mapsto C$ 和 $(A, B) \mapsto C$ 可知 A 在 $(A, B) \mapsto C$ 中是无关的,
于是 $\mathcal{F}_c = \{A \mapsto (B, C), B \mapsto C\}$

- ③ 根据 $A \mapsto (B, C)$ 和 $B \mapsto C$ 可知 C 在 $A \mapsto (B, C)$ 中是无关的, 于是 $\mathcal{F}_c = \{A \mapsto B, B \mapsto C\}$

第二次循环中 $\mathcal{F}_c$ 不会发生改变, 故算法终止, 得到 $\mathcal{F}_c = \{A \mapsto B, B \mapsto C\}$

正则覆盖不一定是唯一的.

例如 $\mathcal{F} = \{A \mapsto (B, C), B \mapsto (A, C), C \mapsto (A, B)\}$ 至少有以下四种正则覆盖:

- $\mathcal{F}_c = \{A \mapsto B, B \mapsto C, C \mapsto A\}$
- $\mathcal{F}_c = \{A \mapsto C, C \mapsto B, B \mapsto A\}$
- $\mathcal{F}_c = \{A \mapsto B, B \mapsto (A, C), C \mapsto B\}$
- $\mathcal{F}_c = \{A \mapsto C, C \mapsto (A, B), B \mapsto C\}$

3.2.6 保持依赖

给定函数依赖集 $\mathcal{F}$

设 $\mathcal{F}_1, \dots, \mathcal{F}_m$ 是能够被高效检查的函数依赖集

若 $\text{cl}(\mathcal{F}_1 \cup \dots \cup \mathcal{F}_m) = \text{cl}(\mathcal{F})$,

则我们称 $\mathcal{F}_1, \dots, \mathcal{F}_m$ 为**保持依赖的分解** (dependency-preserving decomposition)

一种测试某分解是否保持依赖的算法如下:

```

compute  $F^+$ ;
for each schema  $R_i$  in  $D$  do
  begin
     $F_i :=$  the restriction of  $F^+$  to  $R_i$ ;
  end
 $F' := \emptyset$ 
for each restriction  $F_i$  do
  begin
     $F' = F' \cup F_i$ 
  end
compute  $F'^+$ ;
if ( $F'^+ = F^+$ ) then return (true)
  else return (false);

```

Figure 7.10 Testing for dependency preservation.

但上述算法需要计算 $\text{cl}(\mathcal{F})$, 因此开销很大.

我们有两种替代方法:

- ① 验证 $\mathcal{F}$ 中的每个函数依赖 $\alpha \mapsto \beta$, 看它是否在 $\mathcal{F}_1 \cup \dots \cup \mathcal{F}_m$ 中被保持.
为此我们计算 $\mathcal{F}_1 \cup \dots \cup \mathcal{F}_m$ 下 α 的闭包 $\text{cl}(\alpha)$
若 $\beta \subseteq \text{cl}(\alpha)$, 则函数依赖 $\alpha \mapsto \beta$ 在 $\mathcal{F}_1 \cup \dots \cup \mathcal{F}_m$ 中被保持.
当且仅当 $\mathcal{F}$ 中的每个函数依赖 $\alpha \mapsto \beta$ 都在 $\mathcal{F}_1 \cup \dots \cup \mathcal{F}_m$ 中被保持时,
我们称 $\mathcal{F}_1, \dots, \mathcal{F}_m$ 为 $\mathcal{F}$ 的保持依赖的分解.
- ② 对于 $\mathcal{F}$ 中的每个函数依赖 $\alpha \mapsto \beta$, 都执行下面的过程:

```

result =  $\alpha$ 
repeat
    for each  $R_i$  in the decomposition
         $t = (result \cap R_i)^+ \cap R_i$ 
         $result = result \cup t$ 
until (result does not change)

```

3.3 范式

3.3.1 Boyce-Codd 范式

Boyce-Codd 范式 (Boyce-Codd Normal Form, BCNF)

它可以消除基于函数依赖能够发现的所有冗余，不过可能还保留着其他类型的冗余。

关于函数依赖集 $\mathcal{F}$ 的关系模式 $r(R)$ 符合 BCNF 的条件是：

对于 $\mathcal{F}$ 中的任意函数依赖 $\alpha \mapsto \beta$ (其中 $\alpha, \beta \subseteq \mathbb{R}$)，至少满足以下一项：

- ① $\alpha \mapsto \beta$ 是平凡的函数依赖 (即 $\beta \subseteq \alpha$)
- ② α 是模式 R 的一个超键

若一个数据库设计的所有模式都符合 BCNF，则我们称该数据库设计符合 BCNF。

在 3.1.8 中我们研究过以下分解：

```

inst_dept(ID, name, salary, dept_name, building, budget)
      ↓
instructor(ID, name, dept_name, salary)
department(dept_name, building, budget)

```

原来的 inst_dept 模式不符合 BCNF，

例如 $dept_name \mapsto budget$ 是一个非平凡的函数依赖，

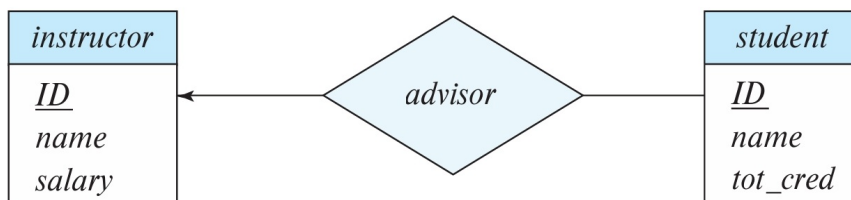
但 dept_name 不是 inst_dept 模式的超键。

分解得到的 instructor 和 department 模式符合 BCNF：

- 在 instructor 模式中，(name, dept_name, salary) 的任意组合上不存在非平凡的函数依赖；
- 在 department 模式中，(building, budget) 的任意组合上不存在非平凡的函数依赖。

现考虑另一个例子。

在 3.1.6 节的大学数据库中，我们要求一名学生只能有一位导师：



现在我们转而要求“一名学生可以有不止一位导师，但是对于一个给定的系不超过一位”：

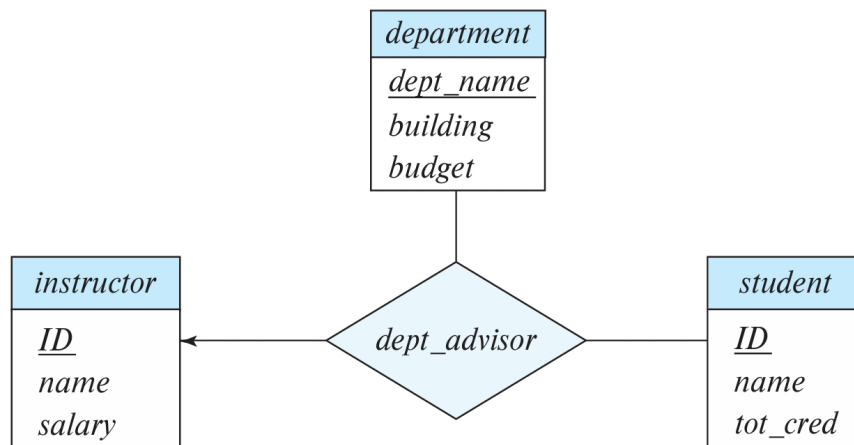


Figure 7.6 The *dept_advisor* relationship set.

其中 *dept_advisor* 联系集导出的模式为:

$\text{dept_advisor}(\text{s_id}, \text{i_id}, \text{dept_name})$

它具有以下函数依赖:

- $\text{i_id} \mapsto \text{dept_name}$ (一位教师只能在单个系中担任导师)
- $(\text{s_id}, \text{dept_name}) \mapsto \text{i_id}$ (对于一个给定的系, 一名学生至多有一位导师)

每次当一位教师 (*i_id*) 参与到一个 *dept_advisor* 联系中时, 我们都不得不重复一次系名 (*dept_name*)
因此 *dept_advisor* 模式不符合 BCNF (其根源是 $\text{i_id} \mapsto \text{dept_name}$ 中的 *i_id* 不是超键)
根据 BCNF 分解规则, 我们得到:

$(\text{i_id}, \text{dept_name})$
 $(\text{s_id}, \text{i_id})$

值得注意的是, 函数依赖 $\text{i_id} \mapsto \text{dept_name}$ 可以在单个关系上强制实施,
但函数依赖 $(\text{s_id}, \text{dept_name}) \mapsto \text{i_id}$ 不能在单个关系上强制实施, 必须在连接上述两个关系后进行检查.
由于我们的设计并不允许在没有连接的情况下强制实施这种函数依赖, 所以我们的设计是非保持依赖的.
为减少检查函数依赖的代价, 我们可使用保持依赖的范式——第三范式.

3.3.2 第三范式

第三范式 (Third Normal Form, 3NF) 的约束比 Boyce-Codd 范式要弱一些.
它允许存在左侧不是超键的特定的非平凡函数依赖.

关于函数依赖集 $\mathcal{F}$ 的关系模式 $r(R)$ 符合 3NF 的条件是:

对于 $\mathcal{F}$ 中的任意函数依赖 $\alpha \mapsto \beta$ (其中 $\alpha, \beta \subseteq \mathbb{R}$), 至少满足以下一项:

- ① $\alpha \mapsto \beta$ 是平凡的函数依赖 (即 $\beta \subseteq \alpha$)
- ② α 是模式 R 的一个超键
- ③ $\beta - \alpha$ 中的每个属性都被包含在 R 的某个候选键中 (回想一下, 候选键是小到不能再小的超键)
注意这不是说单个候选键必须包含 $\beta - \alpha$ 中的所有属性,
 $\beta - \alpha$ 中的不同属性可能被包含于不同的候选键中.
这种 "放松" 有助于确保每一个模式都能保持依赖地分解到 3NF

若一个数据库设计的所有模式都符合 3NF, 则我们称该数据库设计符合 3NF.

3NF 的优点是: 总可以在不牺牲无损性或依赖保持性的前提下得到 3NF 的设计.

然而 3NF 也有缺点: 可能不得不用空值来表示数据项之间的某些可能有意义的联系, 并且存在信息重复的问题.

考虑 3.3.1 中的 *dept_advisor* 模式:

$\text{dept_advisor}(\text{s_id}, \text{i_id}, \text{dept_name})$

它具有以下函数依赖:

- $i_id \mapsto dept_name$ (一位教师只能在单个系中担任导师)
- $(s_id, dept_name) \mapsto i_id$ (对于一个给定的系, 一名学生至多有一位导师)

对于函数依赖 $i_id \mapsto dept_name$ 来说, $dept_name$ 包含在候选键 $(s_id, dept_name)$ 中. 因此 $dept_advisor$ 模式满足 3NF.

我们应用函数依赖进行数据库设计的目标是:

- ① BCNF
- ② 无损性
- ③ 依赖保持性

当不存在保持依赖的 BCNF 设计时, 我们必须在 BCNF 和 3NF 之间进行权衡.

3.3.3 BCNF 的检测

检测关系模式 $r(R)$ 是否满足 BCNF 可以做如下简化:

- 为检查一个非平凡的依赖 $\alpha \mapsto \beta$ 是否违反 BCNF, 可以计算 α 的属性闭包 $cl(\alpha)$ 并验证 $cl(\alpha)$ 是否包含了 R 中的所有属性, 也就是说, 验证 α 是否是 R 的一个超码.
- 为检查关系模式 $r(R)$ 是否属于 BCNF, 只需检查给定集合 $\mathcal{F}$ 中的依赖是否违反 BCNF 就足够了, 而不用检查 $\mathcal{F}$ 中的所有依赖. 我们可以证明: 如果 $\mathcal{F}$ 中没有依赖违反 BCNF, 那么在 $cl(\mathcal{F})$ 中也不会有依赖会违反 BCNF.

遗憾的是, 当一个关系模式被分解时, 后一个过程就不再适用了.

换言之, 当我们判定 R 上的一个分解中的关系模式 R_i 是否违反 BCNF 时, 只用 $\mathcal{F}$ 就不够了.

考虑关系模式 (A, B, C, D, E) , 其函数依赖集 $\mathcal{F}$ 包括 $A \mapsto B$ 和 $(B, C) \mapsto D$.

假设 R 被分解成 (A, B) 和 (A, C, D, E) .

现在 $\mathcal{F}$ 中没有一个函数依赖只包含来自 (A, C, D, E) 的属性, 所以我们也可能会误认为它满足 BCNF.

实际上, 在 $\mathcal{F}$ 中存在一个函数依赖 $(A, C) \mapsto D$, 这表明 (A, C, D, E) 不属于 BCNF.

所以我们也许需要一个属于 $\mathcal{F}$ 但不属于 $\mathcal{F}$ 的依赖, 来证明一个分解后的关系不属于 BCNF.

BCNF 的另一种可替代测试有时会比计算 $\mathcal{F}$ 中的所有依赖更为简单.

为检查 R 分解后的一个关系模式 R_i 是否属于 BCNF, 我们可用这样的测试:

- 对于 R_i 中属性的每个子集 α , 检查 $\mathcal{F}$ 下 α 的属性闭包 $cl(\alpha)$ 看它是否要么不包含 $R_i - \alpha$ 的任何属性, 要么包含 R_i 的所有属性. 只要 R_i 中有某个属性集 α 违反了上述条件, 则函数依赖 $\alpha \mapsto (cl(\alpha) - \alpha) \cap R_i$ (可以证明它出现在 $\mathcal{F}$ 中) 说明 R 违反了 BCNF.

3.3.4 BCNF 分解算法

若 R 不属于 BCNF, 则我们可通过如下算法将 R 分解为一组 BCNF 模式 $R_1, \dots, R_m$:

```

result := {R};
done := false;
while (not done) do
    if (there is a schema  $R_i$  in result that is not in BCNF)
        then begin
            let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that holds
            on  $R_i$  such that  $\alpha^+$  does not contain  $R_i$  and  $\alpha \cap \beta = \emptyset$ ;
            result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
        end
    else done := true;
end

```

Figure 7.11 BCNF decomposition algorithm.

值得注意的是, 由于我们要求 $\alpha \cap \beta = \emptyset$

故当我们用 $(R_i - \beta)$ 和 (α, β) 去取代模式 R_i 时, 依赖 $\alpha \mapsto \beta$ 成立, 而且 $(R_i - \beta) \cap (\alpha, \beta) = \alpha$
 因此上述算法得到的总是无损分解 (可惜的是, 它具有初始模式 R 规模的指数级别的时间复杂度).

考虑 class 关系模式:

class(course_id, title, dept_name, credits, sec_id, semester, year, building, room_number, capacity, time_slot_id)

一个候选键是 (course_id, sec_id, semester, year)

给定函数依赖集:

$$\begin{aligned} & \text{course_id} \mapsto (\text{title}, \text{dept_name}, \text{credits}) \\ & (\text{building}, \text{room_number}) \mapsto \text{capacity} \\ & (\text{course_id}, \text{sec_id}, \text{semester}, \text{year}) \mapsto (\text{building}, \text{room_number}, \text{time_slot_id}) \end{aligned}$$

应用 BCNF 分解算法:

- ① 注意到函数依赖 $\text{course_id} \mapsto (\text{title}, \text{dept_name}, \text{credits})$ 成立,
 但 course_id 不是超键, 因此 class 不属于 BCNF.
 我们将其分解为:

$$\begin{aligned} & \text{course}(\text{course_id}, \text{title}, \text{dept_name}, \text{credits}) \\ & \text{class_1}(\text{course_id}, \text{sec_id}, \text{semester}, \text{year}, \text{building}, \text{room_number}, \text{capacity}, \text{time_slot_id}) \end{aligned}$$

其中 course 属于 BCNF.

- ② class_1 的一个候选键是 (course_id, sec_id, semester, year)
 注意到函数依赖 $(\text{building}, \text{room_number}) \mapsto \text{capacity}$ 在 class_1 上成立,
 但 $(\text{building}, \text{room_number})$ 不是 class_1 的超键, 因此 class_1 不属于 BCNF.
 我们将其分解为:

$$\begin{aligned} & \text{classroom}(\text{building}, \text{room_number}, \text{capacity}) \\ & \text{section}(\text{course_id}, \text{sec_id}, \text{semester}, \text{year}, \text{building}, \text{room_number}, \text{time_slot_id}) \end{aligned}$$

其中 classroom 和 section 都属于 BCNF.

(注意到函数依赖 $(\text{course_id}, \text{sec_id}, \text{semester}, \text{year}) \mapsto (\text{building}, \text{room_number}, \text{time_slot_id})$ 在 section 上成立)

这样我们就将 class 关系模式分解为 course, classroom, section, 它们都属于 BCNF.

可以验证上述分解是无损的, 而且保持依赖.

3.3.5 3NF 分解算法

以下算法可将关系模式无损且保持依赖地分解到 3NF:

```
let  $F_c$  be a canonical cover for  $F$ ;  
 $i := 0$ ;  
for each functional dependency  $\alpha \rightarrow \beta$  in  $F_c$   
     $i := i + 1$ ;  
     $R_i := \alpha \beta$ ;  
if none of the schemas  $R_j, j = 1, 2, \dots, i$  contains a candidate key for  $R$   
    then  
         $i := i + 1$ ;  
         $R_i :=$  any candidate key for  $R$ ;  
/* Optionally, remove redundant relations */  
repeat  
    if any schema  $R_j$  is contained in another schema  $R_k$   
        then  
            /* Delete  $R_j$  */  
             $R_j := R_i$ ;  
             $i := i - 1$ ;  
until no more  $R_j$ s can be deleted  
return ( $R_1, R_2, \dots, R_i$ )
```

Figure 7.12 Dependency-preserving, lossless decomposition into 3NF.

该算法使用的 $\mathcal{F}_c$ 是函数依赖集 $\mathcal{F}$ 的一个正则覆盖.

它考虑的是集合 $\{R_1, \dots, R_i\}$, 其中 i 的初值为 0 (对应的集合为空集)

考虑 dept_advisor 关系模式:

dept_advisor(s_id, i_id, dept_name)

它具有以下函数依赖:

- $f_1 : i_id \mapsto dept_name$ (一位教师只能在单个系中担任导师)
- $f_2 : (s_id, dept_name) \mapsto i_id$ (对于一个给定的系, 一名学生至多有一位导师)

我们在 3.3.1 节和 3.3.2 节说过上述关系模式不属于 BCNF, 但属于 3NF.

由于函数依赖集 $\mathcal{F} = \{f_1, f_2\}$ 的 f_1, f_2 都不存在无关属性, 故其正则覆盖 $\mathcal{F}_c = \{f_1, f_2\}$

- ① 首先基于 f_1 生成 $R_1(i_id, dept_name)$
 - ② 其次基于 f_2 生成 $R_2(s_id, dept_name, i_id)$
 - ③ 然后发现 R_2 包含了 R 的一个候选键 $(s_id, dept_name)$, 于是不再继续创建关系模式.
 - ④ 最后发现 R_2 包含了 R_1 的所有属性, 因此删去 R_1 , 只保留 R_2 .
-

考虑 class 关系模式:

class(course_id, title, dept_name, credits, sec_id, semester, year, building, room_number, capacity, time_slot_id)

一个候选键是 (course_id, sec_id, semester, year)

给定函数依赖集:

$course_id \mapsto (title, dept_name, credits)$
 $(building, room_number) \mapsto capacity$
 $(course_id, sec_id, semester, year) \mapsto (building, room_number, time_slot_id)$

它恰好是自己的正则覆盖.

应用 3NF 分解算法, 我们得到如下分解:

```
course(course_id, title, dept_name, credits)
classroom(building, room_number, capacity)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
```

上例说明了有些关系模式不仅属于 3NF, 同时还属于 BCNF.

这就引出了生成 BCNF 设计的一种可替代方法:

首先使用 3NF 算法, 然后对于 3NF 设计中任何不属于 BCNF 的模式使用 BCNF 算法对其进行分解.

如果分解结果不是保持依赖的, 则还原到 3NF 设计.

3.3.6 其他范式

(教材 7.3.4 节 & 7.7 节)

The End